İş Parçacığı (Thread) Kullanımının Faydaları

- **Cevap Verebilirlik (Responsiveness)**
  Uygulama, bir iş parçacığı bekleme durumundayken diğer iş parçacıkları sayesinde çalışmaya devam eder.
- **Kaynak Paylaşımı (Resource Sharing)**
  İş parçacıkları aynı adres alanını ve kaynakları paylaşır.
- **Ekonomi (Economy)**
  İş parçacığı oluşturma ve bağlam değiştirme (context switch) maliyeti, işlem (process) oluşturmaya göre çok daha düşüktür.
  Örneğin **Solaris** işletim sisteminde:
  - Thread oluşturma ≈ işlem oluşturmanın **1/30'u**
  - Context switch ≈ işlem değiştirmenin **1/5'i**
- **Ölçeklenebilirlik (Scalability)**
  Çok çekirdekli sistemlerde iş parçacıkları farklı çekirdeklerde paralel çalışabilir.

Çok-Çekirdekli Programlama

Çok çekirdekli sistemler, programcıları çok iş parçacıklı uygulamalar yazmaya zorlamaktadır. Bu durum bazı zorlukları beraberinde getirir:

- **Aktiviteleri bölmek (Dividing Activities)**
  Programın paralel çalışabilecek parçalara ayrılması gerekir.
- **Denge (Balance)**
  İş yükü iş parçacıkları arasında dengeli dağıtılmalıdır.
- **Veriyi Ayırmak (Data Splitting)**
  Verilerin iş parçacıkları arasında uygun şekilde paylaştırılması gerekir.
- **Veri Bağımlılığı (Data Dependency)**
  İş parçacıklarının aynı veriye aynı anda erişmesi sorunlara yol açabilir.
- **Test ve Hata Ayıklama (Testing and Debugging)**
  Çok iş parçacıklı programlarda hataları tespit etmek daha zordur.

fork() ve exec() Sistem Çağrıları

- **fork()**
  Yeni bir işlem (process) oluşturur.
  Çağıran işlem kopyalanır ve yeni bir işlem ortaya çıkar.

Çok iş parçacıklı bir süreçte fork() çağrısının, yalnızca çağıran iş parçacığını mı yoksa tüm iş parçacıklarını mı kopyalayacağı işletim sistemi tasarımına bağlıdır.

- **exec()**
  Mevcut işlemin bellek içeriğini siler ve yerine yeni bir program yükler.
  İşlem kimliği (PID) değişmez, ancak çalışan program tamamen değişir.